МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

федерального государственного бюджетного образовательного учреждения

высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №2

по дисциплине: Поддержка и тестирование программных модулей

Руководитель:

Д.Б.Берестов/И.О.Фамилия/

“__16__”октябрь__2026 г.

Исполнитель:

П.Ю. Воробьёва/И.О.Фамилия/

“__16__”октябрь__2026 г.

Пермь, 2026 г.

Оглавление

Введение **3**
Создание проектов в Visual Studio **4**
Написание кода библиотеки **6**
Модульное тестирование **7**
Разработка интерфейса WPF **8**
Проверка работы программы **13**
Заключение **14**

Введение

В этой практической работе мне нужно было сделать калькулятор на WPF в Visual Studio. Главное условие от преподавателя - вся математика (сложение, вычитание, умножение, деление и возведение в степень) должна быть вынесена в отдельную библиотеку классов. Ещё нужно было написать тесты к этой библиотеке и сделать так, чтобы в интерфейсе было видно не только ответ, но и само введенное выражение.

Создание проектов в Visual Studio

Сначала я открыла Visual Studio и создала новое решение. В нем нужно было сделать три проекта.

Первым я создала WPF-приложение (интерфейс) и назвала его Vorobeva. Платформу везде выбирала .NET 8.0, чтобы не было конфликтов.

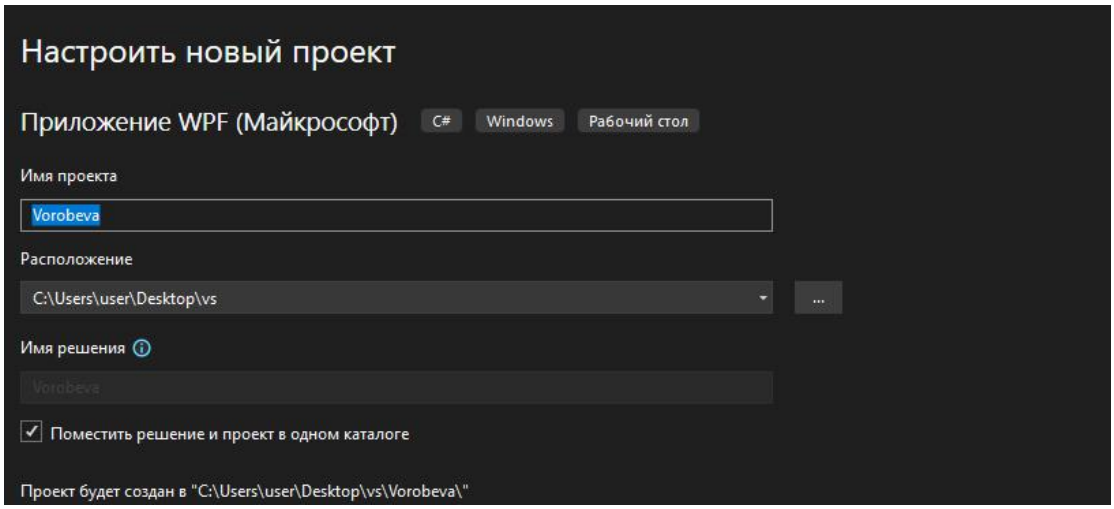


Рисунок 1 - Создание проекта

Вторым проектом я создала библиотеку классов, назвала её CalculatorLogic. Именно тут должен лежать весь код для расчетов.

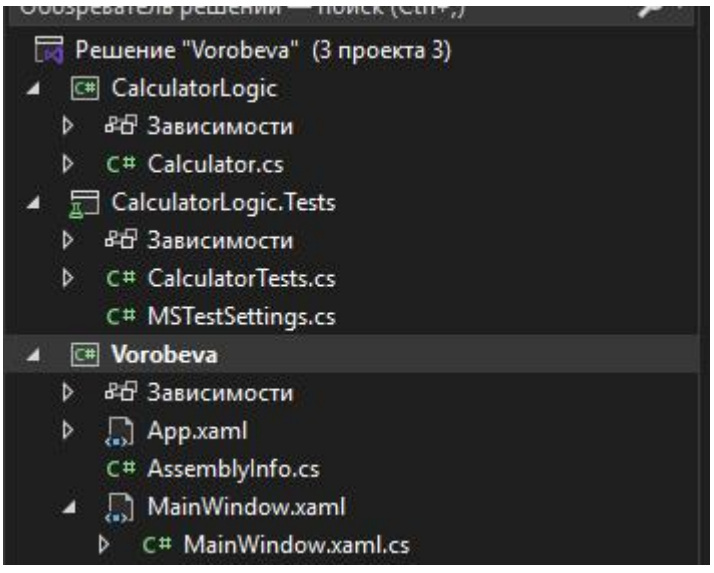


Рисунок 2 - Создание библиотеки классов

Третьим я создала проект для тестов. Выбрала шаблон MSTest и назвала проект CalculatorLogic.Tests.

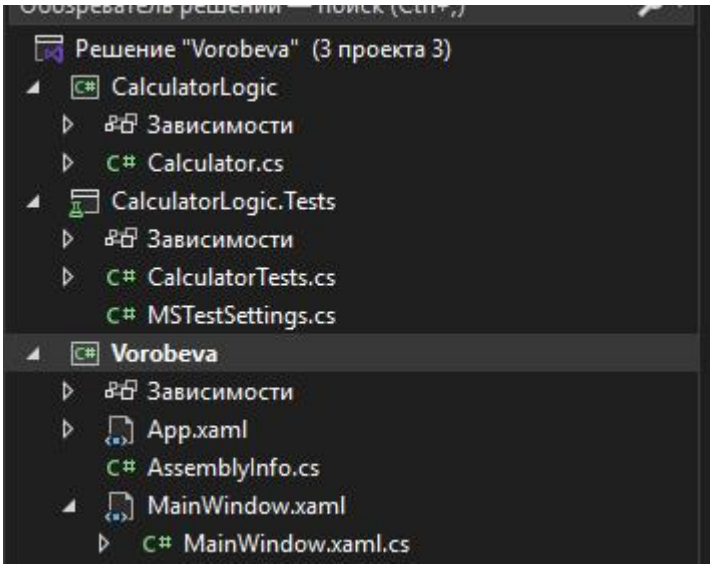
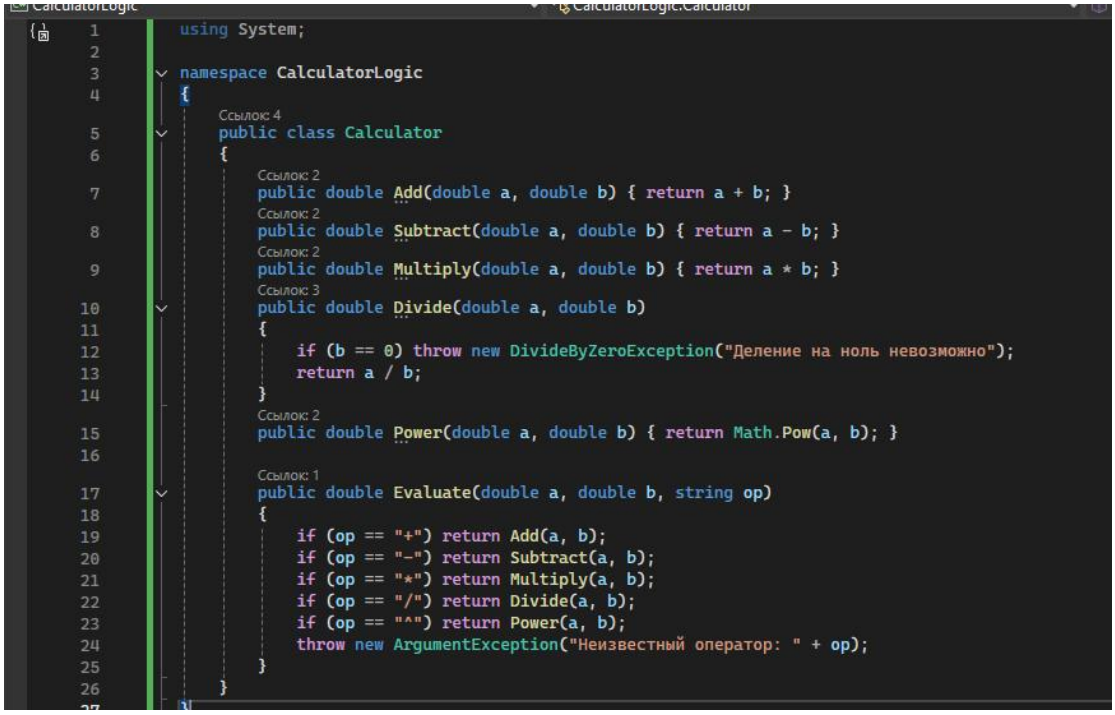


Рисунок 3 - Создание проекта теста MSTest

Чтобы тесты и сам калькулятор видели мою библиотеку, я добавила ссылки. В проектах Vorobeva и CalculatorLogic.Tests зашла в «Зависимости», нажала правой кнопкой на «Проекты» и добавила ссылку на CalculatorLogic.

Написание кода библиотеки

В библиотеке CalculatorLogic я удалила стандартный файл и создала класс Calculator. Написала в нём методы для математики. Отдельно сделала метод Divide (деление), чтобы он выдавал ошибку, если делить на ноль. Возведение в степень сделала через Math.Pow.



```
using System;

namespace CalculatorLogic
{
    Ссылок: 4
    public class Calculator
    {
        Ссылок: 2
        public double Add(double a, double b) { return a + b; }
        Ссылок: 2
        public double Subtract(double a, double b) { return a - b; }
        Ссылок: 2
        public double Multiply(double a, double b) { return a * b; }
        Ссылок: 3
        public double Divide(double a, double b)
        {
            if (b == 0) throw new DivideByZeroException("Деление на ноль невозможно");
            return a / b;
        }
        Ссылок: 2
        public double Power(double a, double b) { return Math.Pow(a, b); }

        Ссылок: 1
        public double Evaluate(double a, double b, string op)
        {
            if (op == "+") return Add(a, b);
            if (op == "-") return Subtract(a, b);
            if (op == "*") return Multiply(a, b);
            if (op == "/") return Divide(a, b);
            if (op == "^") return Power(a, b);
            throw new ArgumentException("Неизвестный оператор: " + op);
        }
    }
```

Рисунок 4 - Код класса Calculator в библиотеке

Модульное тестирование

Для проверки я написала 6 тестов в файле CalculatorTests.cs. Тестировала каждую операцию, а еще отдельно проверила, что при делении на ноль действительно вылетает правильное исключение. Потом я открыла «Обозреватель тестов» и нажала «Запустить все». Все 6 тестов стали зелеными, значит, библиотека написана без ошибок и всё считает правильно.

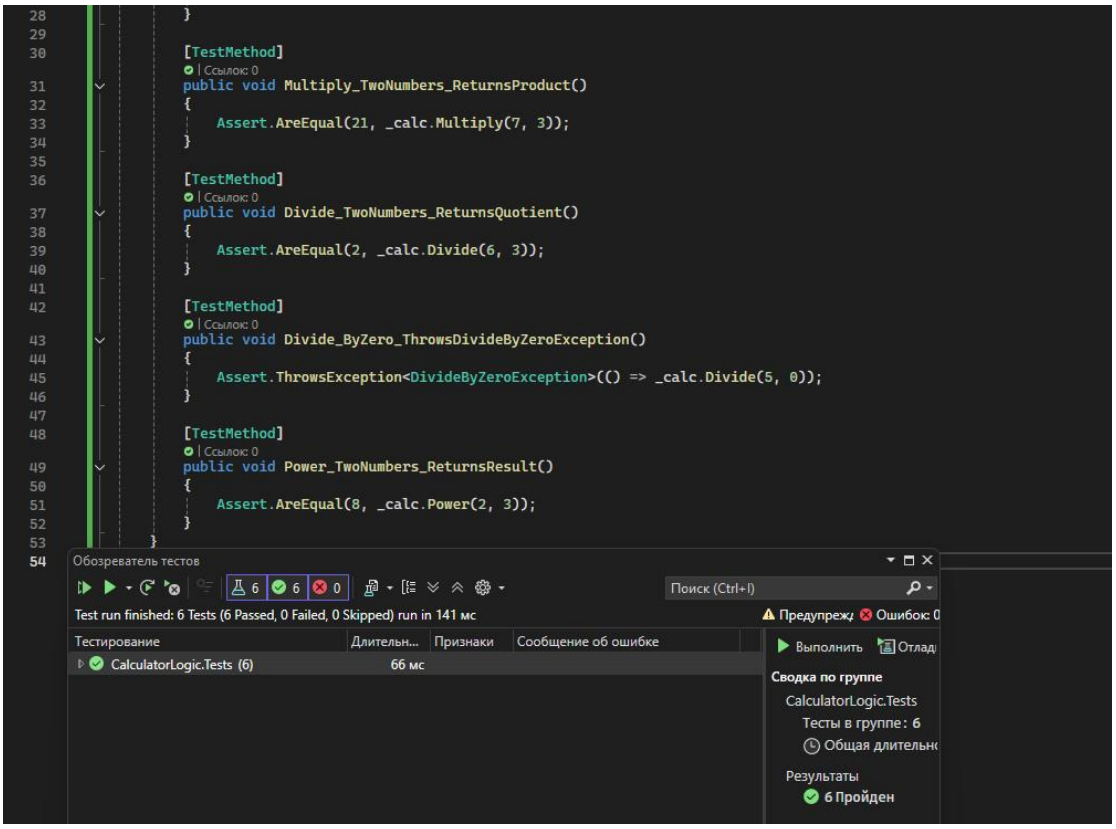
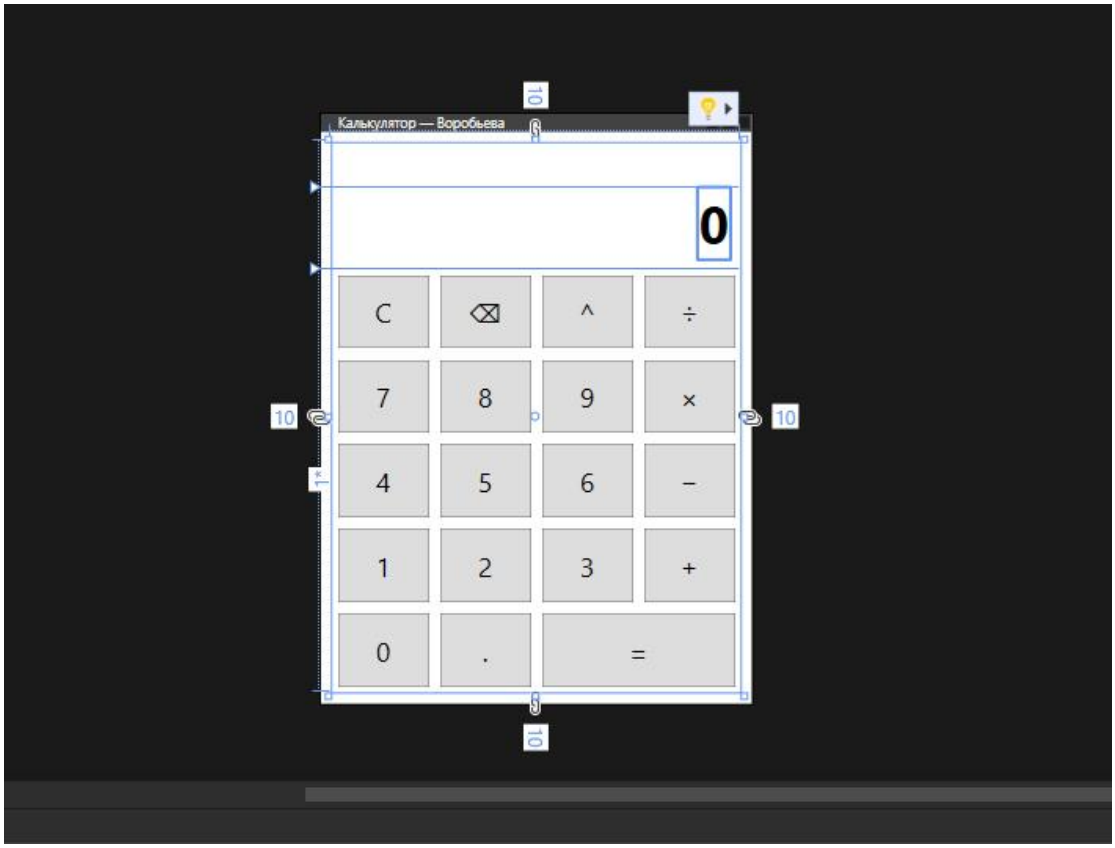


Рисунок 5 - Код и успешное прохождение тестов

Разработка интерфейса WPF

В проекте *Vorobeva* я сделала интерфейс. В файле MainWindow.xaml нарисовала сетку из кнопок и два текстовых поля сверху. Верхнее поле маленькое и серое - для ввода выражения. Нижнее поле большое - для результата.



Риунок 6 - XAML-разметка калькулятора

Код XAML

```
<Window x:Class="Vorobeva.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Калькулятор — Воробьева" Height="520" Width="380"
        WindowStartupLocation="CenterScreen">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Name="ExpressionText"
HorizontalAlignment="Right" FontSize="22" Foreground="Gray"
Margin="10,10,10,0" MinHeight="30"/>
```

```
        <TextBlock Grid.Row="1" Name="ResultText"
HorizontalAlignment="Right" FontSize="46" FontWeight="Bold" Text="0"
Margin="10,0,10,10" MinHeight="60"/>

        <Grid Grid.Row="2">
            <Grid.RowDefinitions>
                <RowDefinition Height="*"/><RowDefinition
Height="*"/><RowDefinition Height="*"/><RowDefinition
Height="*"/><RowDefinition Height="*"/>
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/><ColumnDefinition
Width="*"/><ColumnDefinition Width="*"/><ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>

            <Button Grid.Row="0" Grid.Column="0" Content="C"
Click="ClearButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="0" Grid.Column="1" Content="⌫"
Click="BackspaceButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="0" Grid.Column="2" Content="^" Tag="^"
Click="OperatorButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="0" Grid.Column="3" Content="÷" Tag="/"
Click="OperatorButton_Click" FontSize="24" Margin="5"/>

            <Button Grid.Row="1" Grid.Column="0" Content="7" Tag="7"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="1" Grid.Column="1" Content="8" Tag="8"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="1" Grid.Column="2" Content="9" Tag="9"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="1" Grid.Column="3" Content="×" Tag="*"
Click="OperatorButton_Click" FontSize="24" Margin="5"/>

            <Button Grid.Row="2" Grid.Column="0" Content="4" Tag="4"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="2" Grid.Column="1" Content="5" Tag="5"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="2" Grid.Column="2" Content="6" Tag="6"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="2" Grid.Column="3" Content="−" Tag="-"
Click="OperatorButton_Click" FontSize="24" Margin="5"/>

            <Button Grid.Row="3" Grid.Column="0" Content="1" Tag="1"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="3" Grid.Column="1" Content="2" Tag="2"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="3" Grid.Column="2" Content="3" Tag="3"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="3" Grid.Column="3" Content="+" Tag="+"
Click="OperatorButton_Click" FontSize="24" Margin="5"/>

            <Button Grid.Row="4" Grid.Column="0" Content="0" Tag="0"
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="4" Grid.Column="1" Content="." Tag="."
Click="DigitButton_Click" FontSize="24" Margin="5"/>
            <Button Grid.Row="4" Grid.Column="2" Grid.ColumnSpan="2"
Content="=" Click="EqualsButton_Click" FontSize="24" Margin="5"/>
        </Grid>
    </Grid>
</Window>
```

Код CS

```
using System;
using System.Globalization;
using System.Windows;
using System.Windows.Controls;
using CalculatorLogic;

namespace Vorobeva
{
    public partial class MainWindow : Window
    {
        private Calculator _calculator = new Calculator();
        private double _previousValue = 0;
        private string _pendingOperator = "";
        private string _currentInput = "0";
        private bool _newInput = true;
        private string _expressionText = "";

        public MainWindow()
        {
            InitializeComponent();
            UpdateDisplay();
        }

        private void UpdateDisplay()
        {
            ExpressionText.Text = _expressionText;
            ResultText.Text = _currentInput;
        }

        private void DigitButton_Click(object sender, RoutedEventArgs e)
        {
            Button btn = (Button)sender;
            string digit = btn.Tag.ToString();

            if (_newInput || _currentInput == "0")
            {
                _currentInput = digit == "." ? "0." : digit;
                _newInput = false;
            }
            else
            {
                if (digit == "." && _currentInput.Contains(".")) return;
                _currentInput += digit;
            }
            UpdateDisplay();
        }

        private void OperatorButton_Click(object sender, RoutedEventArgs e)
        {
            Button btn = (Button)sender;
            string op = btn.Tag.ToString();

            if (_pendingOperator != "" && !_newInput) ExecuteCalculation();
            else _previousValue = double.Parse(_currentInput,
CultureInfo.InvariantCulture);

            _pendingOperator = op;
            _newInput = true;
            _expressionText = _previousValue.ToString() + " " +
GetSymbol(op);
```

```
            UpdateDisplay();
        }

        private void EqualsButton_Click(object sender, RoutedEventArgs e)
        {
            if (_pendingOperator == "" || _newInput) return;

            string expressionBefore = _previousValue.ToString() + " " +
GetSymbol(_pendingOperator) + " " + _currentInput + " =";
            ExecuteCalculation();
            _expressionText = expressionBefore;
            _pendingOperator = "";
            _newInput = true;
            UpdateDisplay();
        }

        private void ExecuteCalculation()
        {
            try
            {
                double current = double.Parse(_currentInput,
CultureInfo.InvariantCulture);
                double result = _calculator.Evaluate(_previousValue,
current, _pendingOperator);
                _previousValue = result;
                _currentInput = result.ToString("G15",
CultureInfo.InvariantCulture);
            }
            catch (DivideByZeroException)
            {
                MessageBox.Show("Деление на ноль невозможно", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Warning);
                ClearAll();
            }
        }

        private string GetSymbol(string op)
        {
            if (op == "+") return "+";
            if (op == "-") return "−";
            if (op == "*") return "×";
            if (op == "/") return "÷";
            if (op == "^") return "^";
            return op;
        }

        private void ClearButton_Click(object sender, RoutedEventArgs e)
{ ClearAll(); }

        private void ClearAll()
        {
            _currentInput = "0";
            _previousValue = 0;
            _pendingOperator = "";
            _expressionText = "";
            _newInput = true;
            UpdateDisplay();
        }

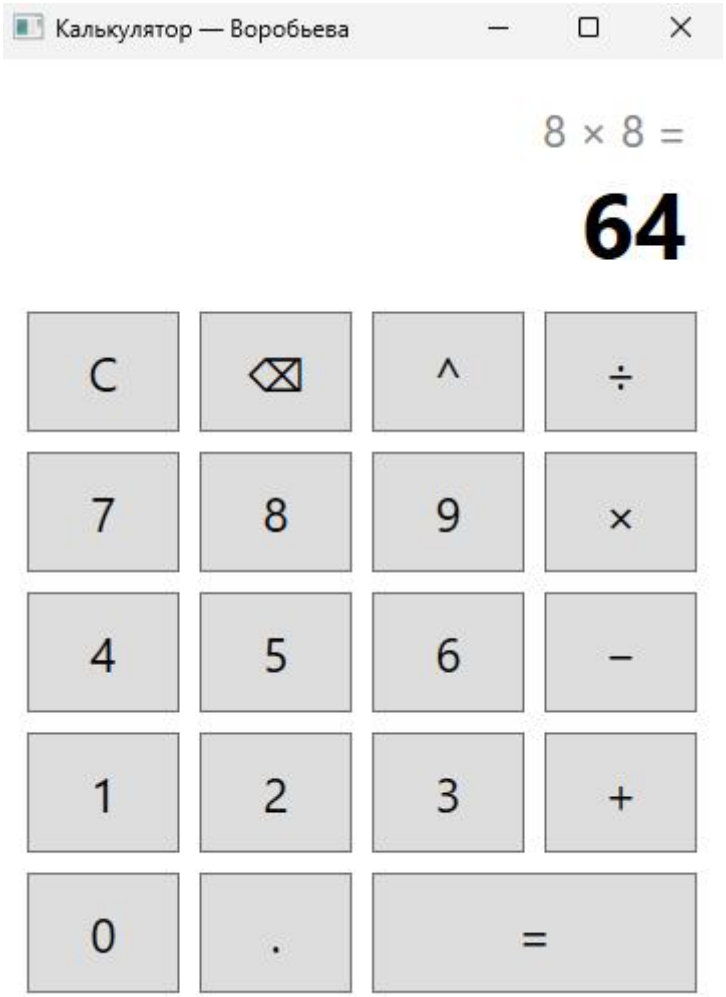
        private void BackspaceButton_Click(object sender, RoutedEventArgs
e)
        {
            if (_newInput) return;
```

```
            if (_currentInput.Length <= 1) _currentInput = "0";
            else _currentInput = _currentInput.Substring(0,
_currentInput.Length - 1);
            UpdateDisplay();
        }
    }
}
```

Логику кнопок я написала в MainWindow.xaml.cs. Сделала так: когда нажимаешь цифры, они вводятся в нижнее поле. Когда нажимаешь оператор (например, умножить), введенное число и знак переходят в верхнее поле. А когда нажимаешь «=», программа берет числа из памяти, вызывает метод Evaluate из библиотеки CalculatorLogic и выводит ответ в нижнее поле. Поэтому само выражение не исчезает, а остается наверху.

Проверка работы программы

Я нажала F5, запустила калькулятор и ввела 8 * 8 =. В верхней строке осталось написано 8 * 8 =, а в нижней появился ответ 64. Всё работает как надо, операция видна.



Рисинок 7 - Проверка работы программы (виден ввод и результат)

Также я проверила деление на ноль: если ввести 5 ÷ 0 =, то выскакивает окошко с предупреждением, программа не ломается.

Заключение

В ходе выполнения данной практической работы я разработала приложение «Калькулятор» на платформе WPF с использованием среды MS Visual Studio. Все поставленные в начале работы задачи были полностью выполнены.

В первую очередь я создала правильную структуру решения, состоящую из трех проектов. Мне удалось реализовать главное требование задания: вынести всю математическую логику в отдельную библиотеку классов CalculatorLogic, а сам интерфейс оставить в проекте Vorobeva. Благодаря этому код получился структурированным, программа, которая отвечает за внешний вид, никак не связана с тем, как именно происходят вычисления. В библиотеке были реализованы все необходимые операции: сложение, вычитание, умножение, деление и возведение в степень. Отдельное внимание я уделила обработке ошибок. Например, при попытке деления на ноль программа выдает понятное предупреждение, а не просто вылетает с ошибкой.

Кроме того, я научилась писать модульные тесты с использованием фреймворка MSTest. Это оказалось очень удобно: когда я вносила изменения в код, я сразу запускала тесты через «Обозреватель тестов» и видела, что ничего не сломалось. Все тесты успешно прошли проверку, что подтверждает правильность работы математических методов.

Отдельным этапом для меня стала разработка самого интерфейса в XAML. Я сделала его похожим на стандартный калькулятор в телефоне. В итоге интерфейс получился удобным и понятным.

Таким образом, в результате практической работы я закрепила свои знания языка C#, изучила принципы объектно-

ориентированного программирования на практике, разобралась с технологией WPF и основами тестирования программного обеспечения. Все цели были достигнуты, задание выполнено в полном объеме.